

REVISÃO PRÉ-PROVA

Programação Backend - 5 exercícios focados nas principais dificuldades

Objetivo da revisão

Esta lista concentra os pontos que mais exigiram atenção nas correções da turma: limites em condicionais, laços com acumuladores, segundo percurso em arrays, integração entre arrays e objetos, e funções/modularização.

Orientação: execute cada solução no terminal e faça os testes indicados. O objetivo é compreender a estrutura usada, não apenas chegar ao resultado.

EXERCICIO 01 - Classificação com limites de faixa

Condicionais e testes de fronteira

Um sensor mede a temperatura de um forno industrial. O sistema deve classificar a leitura sem deixar valores de fronteira fora das faixas.

O programa deve:

- ☐ Solicitar a temperatura pelo terminal.
- ☐ Até 60 °C: exibir NORMAL.
- ☐ De 61 °C até 80 °C: exibir ATENÇÃO.
- ☐ Acima de 80 °C: exibir CRÍTICA.
- ☐ Exibir também a temperatura informada.
- ☐ Testar obrigatoriamente com 60, 61, 80 e 81 °C.

Arquivo	rev01_temperatura.js
Ponto de atenção	Atenção aos operadores >=, <=, > e <. Os valores 60, 61, 80 e 81 ajudam a descobrir erros de limite.

EXERCÍCIO 02 - Média com laço e acumulador

for, acumulador e média

Uma linha de produção registrou a quantidade de peças com defeito em 6 inspeções. O programa deve ler os valores e calcular o total e a média.

O programa deve:

- ☐ Criar um acumulador iniciado em zero.
- ☐ Usar um laço for para solicitar exatamente 6 valores.
- ☐ Somar cada valor ao acumulador.
- ☐ Ao final, calcular a média.
- ☐ Exibir total e média.
- ☐ Não repetir manualmente seis comandos de entrada.

Arquivo	rev02_media.js
Ponto de atenção	O acumulador deve existir antes do laço. A média deve ser calculada somente depois que todas as entradas forem somadas.

EXERCICIO 03 - Cadastro e segunda leitura do array**Array, push(), length e segundo percurso**

Cadastre 5 nomes de operadores e, depois de terminar o cadastro, faça uma segunda passagem pelo array para exibir a lista numerada.

O programa deve:

- ☐ Criar um array vazio.
- ☐ Usar um laço para solicitar 5 nomes.
- ☐ Usar push() dentro do laço de cadastro.
- ☐ Depois do cadastro, criar outro laço para percorrer o array.
- ☐ Usar length no segundo laço.
- ☐ Exibir no formato: 1 - Nome, 2 - Nome, etc.

Arquivo	rev03_operadores.js
Ponto de atenção	Não faça a listagem manual. O segundo laço deve percorrer o array usando <code>i < lista.length</code> .

EXERCÍCIO 04 - Array de objetos com decisão

Objetos, array e condição integrada

Um almoxarifado precisa cadastrar 4 materiais. Cada material possui nome, quantidade atual e estoque mínimo. Ao final, o programa deve indicar quais precisam de reposição.

O programa deve:

- ☐ Criar um array vazio.
- ☐ Usar um laço para cadastrar 4 materiais.
- ☐ Criar um objeto em cada repetição com nome, quantidade e estoqueMinimo.
- ☐ Adicionar cada objeto ao array com push().
- ☐ Depois do cadastro, percorrer o array.
- ☐ Se quantidade < estoqueMinimo, exibir REPOR ESTOQUE; caso contrário, ESTOQUE OK.
- ☐ Exibir nome, quantidade, estoque mínimo e situação.

Arquivo	rev04_estoque.js
Ponto de atenção	O objeto representa um material; o array representa a coleção de materiais. A condição deve usar as propriedades do objeto atual.

EXERCÍCIO 05 - Funções e modularização

Parâmetros, return, require()
e module.exports

Crie um pequeno sistema modular para calcular o custo de manutenção de uma máquina usando dois arquivos.

O programa deve:

- ☐ Criar a pasta rev05_manutencao.
- ☐ Criar funcoesManutencao.js com calcularMaoDeObra(horas), usando R\$ 80,00 por hora.
- ☐ Criar calcularTotal(valorPecas, horas), somando peças e mão de obra.
- ☐ Criar verificarGarantia(meses), retornando EM GARANTIA quando meses ≤ 6 e FORA DA GARANTIA nos demais casos.
- ☐ Exportar as três funções com module.exports.
- ☐ Criar app.js e importar o módulo completo com require('./funcoesManutencao').
- ☐ Solicitar nome da máquina, valor das peças, horas e meses desde a última manutenção.
- ☐ Chamar as funções pelo objeto importado e exibir relatório final completo.

Arquivo	rev05_manutencao/app.js + rev05_manutencao/funcoesManutencao.js
Ponto de atenção	A função deve receber dados por parâmetros e devolver o resultado com return. O app.js deve cuidar das entradas e do relatório.